

Conventional Commits

Types de commits

La spécification impose `feat` et `fix`, mais permet l'utilisation d'autres types selon les conventions de votre équipe :

- `feat` : Introduit une nouvelle fonctionnalité.
- `fix` : Corrige un bug.
- `build` : Modifications affectant le système de build ou les dépendances externes.
- `chore` : Tâches de maintenance (ne modifie pas le code source ni les tests).
- `ci` : Modifications des fichiers et scripts de configuration CI (Continuous Integration).
- `docs` : Modifications de la documentation uniquement.
- `style` : Changements n'affectant pas la signification du code (espaces, formatage, points-virgules manquants).
- `refactor` : Changement de code qui n'est ni une correction de bug, ni l'ajout d'une fonctionnalité.
- `perf` : Changement améliorant les performances.
- `test` : Ajout de tests manquants ou correction de tests existants.

Structure d'un commit

Le message de commit doit être structuré de la manière suivante :

```
<type>[optional scope]: <description>
[optional body]
[optional footer(s)]
```

Exemple:

```
feat(api)!: ajouter la gestion de l'authentification OAuth2
Implémentation complète du flux d'authentification pour permettre aux utilisateurs
de se connecter via des fournisseurs tiers. Ce changement remplace l'ancienne
méthode d'authentification par mot de passe simple.
BREAKING CHANGE: La méthode d'authentification par mot de passe simple est retirée.
Les clients doivent migrer vers OAuth2.
Refs: #123, #456
Reviewed-by: Jean Dupont
```

Le Scope (Portée)

Le **scope** est une information facultative qui permet d'apporter davantage de contexte à votre commit.

- **Rôle** : Il sert à décrire la section spécifique de votre base de code qui est affectée par le changement (par exemple : un module, une bibliothèque ou un composant spécifique).
- **Syntaxe** : Il doit être composé d'un nom entouré de parenthèses et placé juste après le type (et avant le signe deux-points).
- **Exemple** : `fix(parser): ...` (ici, `parser` est le scope).

Le Body (Corps)

Le **body** est un champ libre qui fournit des informations supplémentaires sur les changements effectués.

- **Rôle** : Il sert à expliquer le "pourquoi" et le "comment" des modifications, offrant un contexte utile pour les relecteurs ou pour comprendre l'historique plus tard.
 - **Formatage** : Il est optionnel et doit commencer exactement une ligne blanche après la description courte.
 - **Structure** : Il est totalement libre et peut contenir autant de paragraphes que nécessaire, séparés par des retours à la ligne.
-

Le Footer (Pied de page)

Le **footer** est situé à la toute fin du message de commit et sert principalement aux métadonnées ou aux notes importantes.

- **Rôle** : Il est utilisé pour indiquer des changements majeurs (**BREAKING CHANGE**) ou pour ajouter des informations de suivi (type "git trailer"), comme des références à des tickets ou des approbateurs.
 - **Formatage** :
 - Il doit être placé après une ligne blanche, suite au corps du commit.
 - Chaque ligne du pied de page doit commencer par un "token" (nom du champ), suivi d'un séparateur (`:` ou `#`), puis de la valeur associée.
 - Les jetons (tokens) doivent utiliser des tirets (`-`) pour remplacer les espaces, par exemple `Acked-by` , `Reviewed-by` ou `Refs` .
 - **Cas particulier** : Le jeton **BREAKING CHANGE** est une exception et peut être utilisé tel quel pour signaler une rupture de compatibilité majeure.
-

Le point d'exclamation (**!**)

Le **!** sert à attirer immédiatement l'attention sur le fait que le commit introduit une rupture de compatibilité.

- **Emplacement** : Il doit être placé juste avant les deux-points (`:`), après le type ou le scope (par exemple : `feat!: ...` ou `feat(scope)!: ...`).
- **Fonction** : Lorsqu'il est utilisé dans le préfixe, il rend explicite le caractère "breaking" du commit.
- **Conséquence** : Si vous utilisez le **!** , vous pouvez omettre le pied de page **BREAKING CHANGE:** . Dans ce cas, la description du commit elle-même est utilisée pour décrire la nature du changement.
- **Exemple** : `feat(api)!: supprimer le support de l'authentification par token V1`